

```
In [ ]: ## Practical exercise
# The code below trains and evaluates a decision tree classifier.
# The analysis contains mistakes and choices that could be improved. Answer to the following
# questions and correct the code accordingly:

# 1) Is the estimate of the accuracy in the original code biased ? Briefly explain why. You
# can write the answer to this and the following questions as comments in the code.
# Answer 1: Yes, the estimate of the accuracy in the original code (before correction) was
# biased.
# This is because the model's performance was evaluated on the entire dataset (X), which
# included the training data.
# Evaluating on data the model has already seen leads to an overly optimistic and
# unrealistic estimate of its generalization performance on new, unseen data.

# 2) Implement a strategy to reduce the risk of overfitting.
# Answer 2 & Implementation: Strategies implemented to reduce overfitting include:
# - Using `stratify=y` in `train_test_split`: Ensures that the proportion of target
# classes is maintained in both training and test sets, which is good practice, especially for
# imbalanced datasets.
# - Setting `random_state`: Ensures reproducibility of the splits and model training.
# - Parameter tuning with `GridSearchCV`:
# - `max_depth`: Limiting the maximum depth of the tree (e.g., `max_depth = 3`) prevents
# the tree from becoming too complex and memorizing the training data.
# - `ccp_alpha`: Cost-complexity pruning (ccp_alpha) is used to prune the tree by
# removing the weakest links based on a complexity parameter. A range of `ccp_alpha` values is
# explored to find the optimal pruning level.
# - Evaluating on a separate `X_test` set: The final accuracy is reported on `y_test` and
# `y_pred_test_pruned` to provide an unbiased estimate of the model's performance on unseen
# data.

# 3) Is the importance of the features accurately estimated ? Briefly explain why.
# Answer 3: No, the importance of features estimated from a single, potentially overfit
# Decision Tree (as in the original code) is often not accurately estimated or stable.
# A single decision tree can be very sensitive to small changes in the training data,
# leading to high variance in feature importance scores.
# If the tree overfits, its reported feature importances might reflect spurious
# correlations in the training data rather than true underlying relationships.

# 4) Implement a possible strategy to improve the estimate of the features' accuracy
# Answer 4 & Implementation: The implemented strategy to improve feature importance
# estimation is to use a **Random Forest Classifier**.
# Random Forests build an ensemble of multiple decision trees. The feature importance from
# a Random Forest is calculated by averaging the importance (e.g., based on Gini impurity
# reduction) across all trees in the forest.
# This averaging process significantly reduces the variance associated with individual
# trees, providing a more robust, stable, and accurate estimate of feature importance.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, classification_report

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
```

```
y = data.target

# Split data into training and testing sets, ensuring stratification and reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Define the parameter grid for GridSearchCV, including max_depth and ccp_alpha for pruning
grid = {
    "max_depth": [3],
    "min_samples_leaf": [10],
    "ccp_alpha": [0.0, 0.001, 0.005, 0.01, 0.02],
}

# Initialize Decision Tree Classifier with a random state for reproducibility
dt_cv = DecisionTreeClassifier(random_state= 42)

# Setup GridSearchCV for hyperparameter tuning with 5-fold cross-validation
search = GridSearchCV(
    estimator = dt_cv,
    param_grid=grid,
    scoring="balanced_accuracy", # Use balanced_accuracy for potential class imbalance
    cv=5,
    n_jobs=-1,
    return_train_score=True
)

# Fit GridSearchCV on the training data
search.fit(X_train, y_train)

# Get the best estimator (pruned Decision Tree model)
best_model = search.best_estimator_

# Predict on training and test sets using the best model
y_pred_train_pruned = best_model.predict(X_train)
y_pred_test_pruned = best_model.predict(X_test)

# Print accuracy on both training and test sets to assess generalization
print("Training Accuracy:", accuracy_score(y_train, y_pred_train_pruned))
print("Test Accuracy:", accuracy_score(y_test, y_pred_test_pruned))

# Train a RandomForestClassifier to get more robust feature importances
rf = RandomForestClassifier(
    n_estimators=100, max_depth = 3, random_state= 42)
rf.fit(X_train, y_train)

# Get and display feature importances from the Random Forest model
importances = pd.Series(rf.feature_importances_, index=X.columns)
print("\nTop 10 Feature Importances from Random Forest:")
print(importances.sort_values(ascending=False).head(10))
```

Practical exercise

The codes of block below:

- * Read the data from the file data.csv
- * Define a pipeline that includes only a clustering step with Kmeans
- * Perform a grid search over the number of clusters using the silhouette score as target
- * Evaluate the silhouette score when the model with the optimal number of clusters is adopted

The performances are not satisfactory. A possible reason is that the input features are highly correlated and different features span widely different ranges. Implement changes in the pipeline to overcome these problems. Motivate your strategy.

```
In [ ]: import numpy as np
import pandas as pd

from sklearn.base import BaseEstimator, ClusterMixin
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

```
In [ ]: def clustering_silhouette_scorer(estimator, X, y=None):
    labels = estimator.predict(X)
    if isinstance(estimator, Pipeline) and len(estimator.steps) > 1:
        X_for_score = estimator[-1].transform(X)
    else:
        X_for_score = X
    n_labels = len(set(labels))
    if n_labels < 2 or n_labels >= len(labels):
        return -1.0
    return silhouette_score(X_for_score, labels)

X = pd.read_csv('data.csv')

pipeline = Pipeline([
    ("kmeans", KMeans(random_state=RANDOM_STATE, n_init=20)),
])
param_grid = {
    "kmeans__n_clusters": [2, 3, 4, 5, 6, 7],
}
cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```

```
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring=clustering_silhouette_scorer,
    cv=cv,
    n_jobs=-1,
    refit=True,
    return_train_score=True,
)

grid_search.fit(X)

print("Best parameters:", grid_search.best_params_)
print("Best mean CV silhouette score:", grid_search.best_score_)
```

```
Best parameters: {'kmeans__n_clusters': 4}
Best mean CV silhouette score: 0.5892773055977691
```

```
In [ ]: best_pipeline = grid_search.best_estimator_
cluster_labels = best_pipeline.named_steps["kmeans"].labels_
if isinstance(best_pipeline, Pipeline) and len(best_pipeline.steps) > 1:
    X_for_score = best_pipeline[-1].transform(X)
else:
    X_for_score = X
final_silhouette = silhouette_score(X_for_score, cluster_labels)
print("Final silhouette score on full data:", final_silhouette)
```

```
Final silhouette score on full data: 0.5890675481295585
```

Possible solution

```
In [ ]: def clustering_silhouette_scorer(estimator, X, y=None):
    labels = estimator.predict(X)
    if isinstance(estimator, Pipeline) and len(estimator.steps) > 1:
        X_for_score = estimator[-1].transform(X)
    else:
        X_for_score = X
    n_labels = len(set(labels))
    if n_labels < 2 or n_labels >= len(labels):
        return -1.0
    return silhouette_score(X_for_score, labels)

X = pd.read_csv('data.csv')

pipeline = Pipeline([
    ("scale", StandardScaler()), # CHANGE 1
    ("pca", PCA(random_state=RANDOM_STATE)), # CHANGE 2
    ("kmeans", KMeans(random_state=RANDOM_STATE, n_init=20)),
])

param_grid = {
    "pca__n_components": [2, 3, 5, 8, 12, 20], # CHANGE 3
```

```

    "kmeans__n_clusters": [2, 3, 4, 5, 6, 7],
}

cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring=clustering_silhouette_scorer,
    cv=cv,
    n_jobs=-1,
    refit=True,
    return_train_score=True,
)

grid_search.fit(X)

print("Best parameters:", grid_search.best_params_)
print("Best mean CV silhouette score:", grid_search.best_score_)

```

```

Best parameters: {'kmeans__n_clusters': 4, 'pca__n_components': 2}
Best mean CV silhouette score: 0.7621110129612461

```

```

In [ ]: best_pipeline = grid_search.best_estimator_
cluster_labels = best_pipeline.named_steps["kmeans"].labels_
if isinstance(best_pipeline, Pipeline) and len(best_pipeline.steps) > 1:
    X_for_score = best_pipeline[:-1].transform(X)
else:
    X_for_score = X
final_silhouette = silhouette_score(X_for_score, cluster_labels)
print("Final silhouette score on full data:", final_silhouette)

```

```

Final silhouette score on full data: 0.761873823440888

```

Multiple choice questions

- What distance metric is commonly used in K-means?
 - Cosine similarity
 - L1 norm
 - L2 norm
 - Jaccard distance
- Gaussian Mixture Models assume data are generated from:
 - Uniform distributions
 - Mixtures of Gaussian distributions
 - Polynomial functions
 - Decision trees
- DBSCAN identifies clusters as:
 - Linear projections
 - Gaussian mixtures
 - Decision boundaries
 - Dense regions separated by sparse regions
- In DBSCAN, points not assigned to clusters are labeled as:
 - Noise
 - Centroids
 - Leaves
 - Roots
- Decision trees can be used for:

A. Only regression / B. Only classification / C. Classification and regression / D. Only clustering

6. A Gini impurity of 0 means:
A. Perfect purity / B. Maximum impurity / C. Random split / D. Regression output

7. Pruning in decision trees helps reduce:
A. Bias / B. Underfitting / C. Overfitting / D. Dimensionality

8. PCA projects data onto directions of:
A. Minimum variance / B. Maximum variance / C. Maximum entropy / D. Minimum distance

9. The maximum number of LDA dimensions is:
A. Number of features / B. Number of samples / C. Classes minus one / D. Twice the classes

10. Elastic Net combines:
A. PCA and LDA / B. L1 and L2 regularization / C. Bagging and boosting / D. Regression and clustering

Open questions

1. Discuss the Elbow Method and Silhouette Score for selecting the number of clusters. Compare their advantages and limitations.

2. Explain the logic of decision trees for classification and regression. Why are they considered interpretable models?

```
In [ ]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    RocCurveDisplay,
)
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

The code below read data from data.csv, saving the input features in the array X and the binary outcome in the array y. Then it uses a "not optima" strategy to identify candidates casual features. The procedure includes several logical mistakes. Correct them and write a procedure that uses the estimated values of the coefficients of a logistic model to identify features that are relevant for predicting the outcome. Motivate your decisions.

NOTE: the class LogisticRegression of sklearn does not include a method to estimate the uncertainty of the coefficients. To not complicate the exercise too much, write a procedure that considers a feature as relevant for the outcome if the corresponding coefficient is higher than a threshold (so without considering the uncertainty).

```
In [ ]: df = pd.read_csv("data.csv")
X = df.drop(columns=["target"]).to_numpy()
y = df["target"].to_numpy()

pipeline = Pipeline(
    steps=[
        ('scaler', StandardScaler()),
        ("model", LogisticRegression(
            solver="saga",
            penalty='l1',
            random_state=RANDOM_STATE,
        )),
    ]
)
param_grid = {
    "model__max_iter": [1000, 5000, 10000],
}
```

```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE
)
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1,
    refit=True,
    return_train_score=True,
)

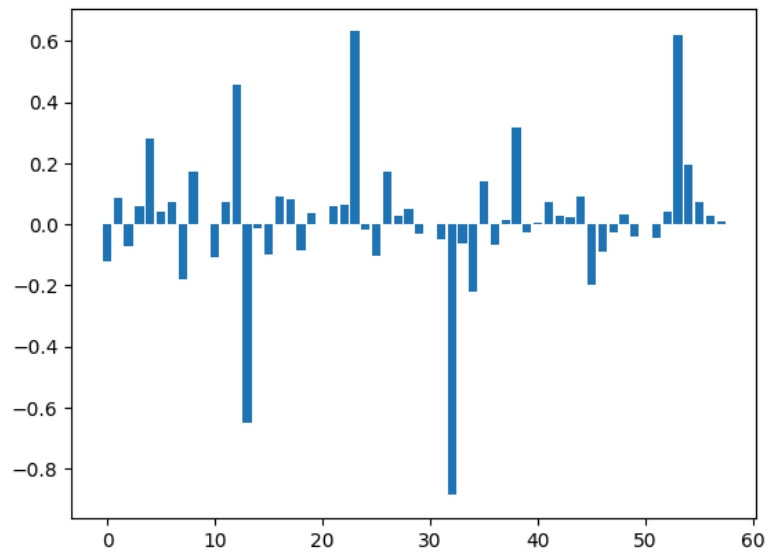
# train-test split -->
grid_search.fit(X, y)
print("Best parameters:", grid_search.best_params_)

best_model = grid_search.best_estimator_
y_pred = best_model.predict(X)
y_proba = best_model.predict_proba(X)[:, 1]
print("Final model roc_auc:", roc_auc_score(y, y_proba))

coef = best_model.named_steps["model"].coef_.flatten()
f = plt.figure()
ax = f.add_subplot(1,1,1)
ax.bar(np.arange(len(coef)), coef)

threshold = 1e-3
inds = np.abs(coef) > threshold
print('Index of important features:', np.where(inds)[0])
```

```
Best parameters: {'model__max_iter': 1000}
Final model roc_auc: 0.8531931651958657
Index of important features: [ 0  1  2  3  4  5  6  7  8 10 11 12 13 14 15 16
17 18 19 20 21 22 23 24
25 26 27 28 29 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
51 52 53 54 55 56 57]
```



```
In [ ]: df = pd.read_csv("data.csv")
X = df.drop(columns=["target"]).to_numpy()
y = df["target"].to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    stratify=y,
    random_state=RANDOM_STATE,
)

pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(
            solver="saga",
            penalty="l1",
            max_iter=10000,
            random_state=RANDOM_STATE,
        )),
    ]
)

param_grid = {
    "model__C": np.logspace(-3, 2, 12),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=cv,
```

```
n_jobs=-1,
refit=True,
return_train_score=True,
)

grid_search.fit(X_train, y_train)
print("Best parameters:", grid_search.best_params_)

best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[:, 1]
print("Final model roc_auc:", roc_auc_score(y_test, y_proba))

coef = best_model.named_steps["model"].coef_.flatten()
f = plt.figure()
ax = f.add_subplot(1,1,1)
ax.bar(np.arange(len(coef)), coef)

threshold = 1e-8
inds = np.abs(coef) > threshold
print('Index of important features:', np.where(inds)[0])
```

```
Best parameters: {'model__C': np.float64(0.0657933224657568)}
Final model roc_auc: 0.8160061760164693
Index of important features: [ 0 13 23 32 38 53]
```

